

项目进度汇报 & 文献介绍

蒙略

2025 年 10 月 8 日

摘要

本文汇报了自动化渗透测试系统的项目进度，并介绍了几个当前代表性自动化渗透测试系统。内容包括新增工具与 workflows 开发进展，以及多个前沿系统的架构设计、性能评估和技术特点分析。

目录

1	项目进度汇报	2
1.1	新增工具/workflows/知识库	2
1.1.1	NVD 漏洞信息搜索	2
1.1.2	CWE 常见弱点分析	3
1.1.3	OWASP Web Security Testing Guide	3
1.1.4	ExploitDB_search	3
2	代表性自动化渗透测试系统介绍	3
2.1	系统对比分析	3
2.2	PentestGPT	3
2.2.1	基本信息	3
2.2.2	概要	4
2.2.3	贡献	4
2.2.4	Malism 框架	5
2.2.5	PENTESTGPT 架构设计	5
2.3	xOffense	7

2.3.1	基本信息	7
2.3.2	概要	7
2.3.3	性能评估	7
2.3.4	自动化渗透测试形式化定义	8
2.3.5	xOffense 架构	8
2.3.6	Why Lora fine-tuning?	9
2.4	MAPTA	10
2.4.1	基本信息	10
2.4.2	概要	10
2.4.3	架构设计	10
2.4.4	通过执行验证	10
2.4.5	基准测试与测试平台	12
2.4.6	可用性	12
2.5	RefPentester	12
2.5.1	基本信息	12
2.5.2	概要	12
2.5.3	现有局限性分析	13
2.5.4	RefPentester 框架设计	13
2.5.5	评估效果	15

1 项目进度汇报

1.1 新增工具/工作流/知识库

1.1.1 NVD 漏洞信息搜索

- **适用 Agent:** 分析者、验证者
- **Action 数量:** 1
- **描述:** 用于漏洞信息检索，需要输入 NVD 编号，可以根据官方 NVD 数据库获取漏洞信息。

1.1.2 CWE 常见弱点分析

- **适用 Agent:** 分析者
- **Action 数量:** 9
- **描述:** CWE (Common Weakness Enumeration) 弱点搜索和分析工具, 提供完整的 CWE 数据库查询功能。

1.1.3 OWASP Web Security Testing Guide

- **适用 Agent:** 执行者、分析者
- **Action 数量:** 1
- **描述:** OWASP Web 安全测试指南集成工具。

1.1.4 ExploitDB_search

- **适用 Agent:** 验证者、执行者
- **Action 数量:** 1
- **描述:** Search for security exploits in ExploitDB database。

2 代表性自动化渗透测试系统介绍

2.1 系统对比分析

2.2 PentestGPT

2.2.1 基本信息

- **地点:** 新加坡、西班牙、奥地利、澳大利亚
- **发布日期:** 2024.6.2
- **论文链接:** <https://arxiv.org/pdf/2308.06782>
- **Github 链接:** <https://github.com/GreyDGL/PentestGPT>

Table 1: Comparison of representative automated pentesting systems and benchmarks (grid-lined). Criteria emphasize architecture, scope, grounding, tools, autonomy, and evaluation.

Work	Architecture (Arch.)	Scope/Phase	Grounding/Memory	Tool Use	Evaluation & Highlights
PentestGPT [7]	SA	Web+Net (multi-phase)	SUM (module summaries)	Parse scans → cmd synth.	USENIX'24 cases/bench; modular pipeline
AutoAttacker [18]	SA (post)	Post-breach, OpSec realism	CTX (in-session)	Shell, Metasploit	Simulated org (Win/Linux); multi-step attacks
PentestAgent [8]	MA	Web focus (ext. assess.)	RAG + MEM	Scanners, PoCs	Bench + HTB; reduced hallucinations via RAG
VulnBot [9]	MA + PTG	Full cycle (recon→exp.)	Phase SUM (PTG state)	Nmap, Nikto, Metasploit	AutoPenBench (30.3% overall; 69.05% subtask); AIPB best-of-six
RefPentester [19]	MA + RFL	Stage-aware triage/exp.	Reflection + knowledge	Std. toolchain	HTB "Sau": +16.7% vs GPT-4o baseline
Perses [21]	MA (multi-LLM)	Privilege escalation	HET (heterogeneous model/task)	Tool-grounded (details in paper)	FreeBSD systems; small-LLM focus
RapidPen [20]	SA	IP→Shell (initial foothold)	MEM + exploit retrieval	Scan→exploit loop	HTB: autonomous shells in minutes; low cost
One-Day Agent [22]	SA	Web (one-day CVEs)	CVE-guided CTX	Browser, tools	87% (GPT-4, with CVE desc.); 7% w/o desc.
HPTSA (Teams) [23]	MA (hier./team)	Web (zero-day)	Planner + experts	Browser, task agents	42% pass@5; 18% pass@1 on 14 real vulns
HackSynth [24]	SA (2-mod.)	CTF (200 tasks)	Planner + summarizer	Sandbox tools	PicoCTF/OTW benchmarks; GPT-4o best
AutoPentest [28]	MA (LangChain)	Black-box (enum→exp.)	Prompting + MEM	Std. scans/exploits	GPT-4o-based prototype; open code
CVE-Bench [26]	Bench	Real web CVEs	—	—	Up to 13% s-rates for SOTA agents
AutoPenBench [10]	Bench	Mixed (Web/Net/CRPT)	—	—	33 tasks; autonomy and milestone scoring
NYU CTF Bench [25]	Bench	CTF (CSAW; 200)	—	Tool-integrated	NeurIPS D&B; open dataset+automation
Our work	MA + PTG	Full cycle (Web+Net)	GBP + MEM	Broad scan/exploit	APB, AIPB, (opt.) CVEB; mid-scale open LLM

Legend: SA=Single-agent; MA=Multi-agent; PTG=Penetration Task Graph; RFL=Reflection; SUM=Summaries; RAG=Retrieval-Augmented Generation; MEM=Explicit memory; CTX=In-session context; HET=Heterogeneity (multi-LLM model/task selection); GBP=Grey-box phase prompting; APB=AutoPenBench [10]; AIPB=AI-Pentest-Benchmark [11]; CVEB=CVE-Bench [26]. "s-rate" denotes success rate.

图 1: 代表性渗透智能体对比表

2.2.2 概要

渗透测试作为确保系统安全的关键工业实践，历来因高度依赖人类专家的专业知识而难以实现自动化。大语言模型在诸多领域展现出显著进步，其涌现能力预示着变革行业的潜力。本研究基于真实渗透测试目标构建了综合性基准测试平台，并以此探索大语言模型在该领域的能力表现。研究发现，大语言模型在渗透测试过程中的特定子任务（如测试工具使用、输出解析及后续行动建议）中表现熟练，但在维护整体测试场景的完整上下文理解方面仍存在困难。

基于这些发现，我们推出了 PENTESTGPT——一个利用大语言模型内在丰富领域知识的智能渗透测试框架。该框架通过精心设计的三个自交互模块，分别处理渗透测试中的独立子任务，有效缓解了上下文丢失带来的挑战。评估结果表明，PENTESTGPT 不仅在基准测试目标中的任务完成度较 GPT-3.5 模型提升 228.6%，更在真实渗透测试目标和 CTF 挑战中验证了其有效性。该框架已在 GitHub 开源，9 个月内获得超过 6,200 个星标，并促进了活跃的社区参与，充分证明了其在学术界与工业界的价值与影响力。

2.2.3 贡献

- 构建了一个包含 HackTheBox 和 VulnHub 这两个领先渗透测试挑战平台测试机器的稳健基准。
- 推出了 PENTESTGPT——一个专为增强 LLMs 在该领域应用而设计的交互式系统。代码仓库：<https://github.com/GreyDGL/PentestGPT>

2.2.4 Malism 框架

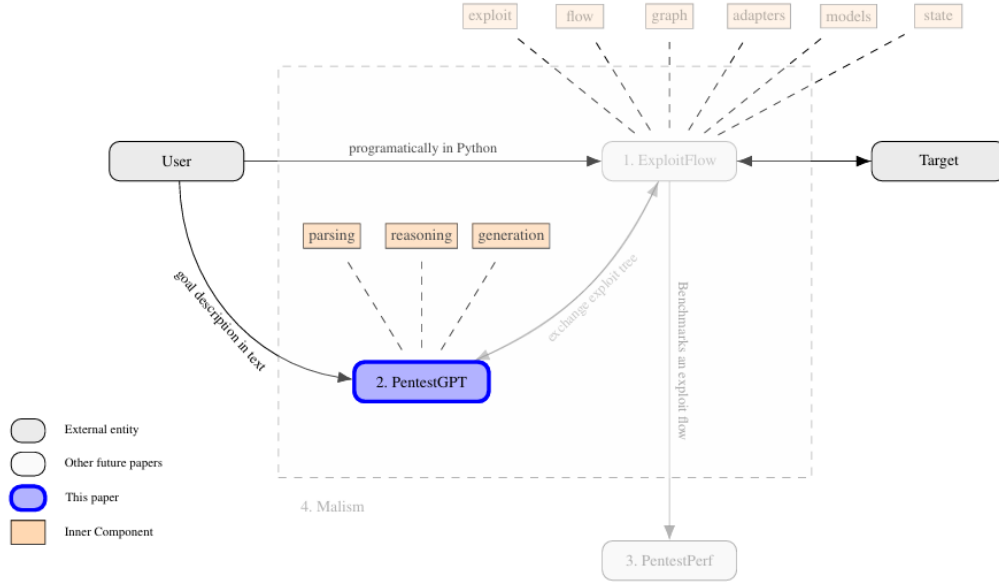


图 2: Malism 架构图

图2展示了全自动渗透测试工具 MALISM 框架架构。该图展示了任意用户使用 MALISM 对给定目标进行渗透测试时可遵循的各种交互流程：

- (1) EXPLOITFLOW——生成安全利用路径（漏洞利用流）的模块化库，能在每个离散操作后捕获被测系统的状态流
- (2) PENTESTGPT——利用 LLM 能力为每个给定离散状态生成测试指导（启发式方法）的测试工具
- (3) PENTESTPERF——综合性渗透测试基准，用于评估渗透测试员和自动化工具在多种测试目标上的性能表现
- (4) MALISM——用于开发全自动渗透测试工具（网络安全认知引擎）的框架

2.2.5 PENTESTGPT 架构设计

定义 1（属性树） 属性树是一种边标记的属性多叉树 $G = (V, E, \lambda, \mu)$ ，其中 V 是节点（或顶点）的集合， E 是有向边的集合， $\lambda: E \rightarrow \Sigma$ 是一个边标记函数，它将字母

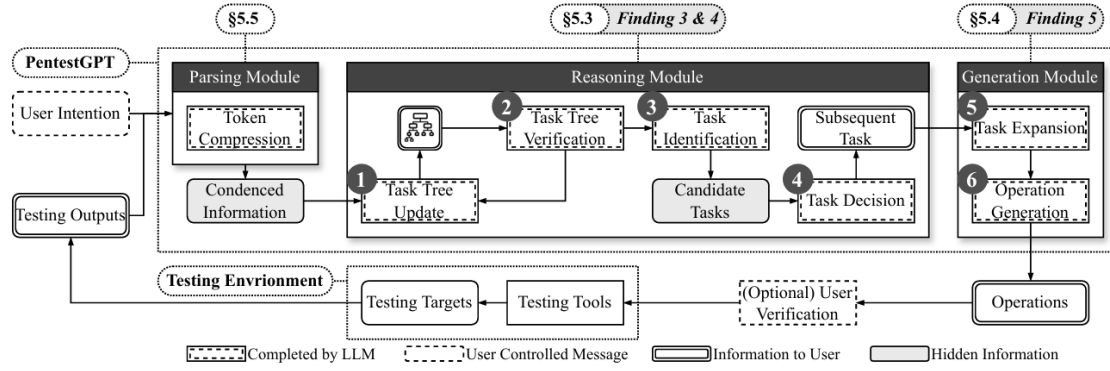


Figure 3: Overview of PENTESTGPT.

表 Σ 中的标签分配给每条边，而 $\mu : (V \cup E) \times K \rightarrow S$ 是一个函数，它将属性（键 K 来自，值 S 来自）的键值对分配给边和节点。

给定属性树的定义，PTT 定义如下：

定义 2（渗透测试任务树） 一个 PTT T 是一个对 (N, A) ，其中：

- (1) N 是以树形结构组织的一组节点。每个节点有一个唯一标识符，并且存在一个称为根的特殊节点，它没有父节点。除根节点外的每个节点有且仅有一个父节点和零个或多个子节点。
- (2) A 是一个函数，它为每个节点 $n \in N$ 分配一组属性 $A(n)$ 。每个属性是一个对 (a, v) ，其中 a 是属性名称， v 是属性值。每个节点的属性集可以不同。

推理模块操作流程 如上图所示，推理模块的操作围绕 PTT 展开四个关键步骤：

1. **初始 PTT 创建：** 模块首先解释用户的目标，以创建一个初始的 PTT，该 PTT 以自然语言格式化。这涉及使用设计的提示词来指导 LLM，这些提示词包含上述 PTT 定义和真实世界的示例。对 LLM 的输出进行解析，以确保树形结构被正确表示。
2. **树信息更新与验证：** 在更新树信息后，对新更新的 PTT 执行验证步骤以确定其正确性。此过程明确检查只有 PTT 的叶节点被修改，这符合渗透测试过程中的原子操作应仅影响最低级别子任务状态的原则。此步骤确认了推理过程的正确性，防止由于 LLM 的幻觉而对整体树结构造成任何潜在更改。如果出现差异，信息将回传给 LLM 进行纠正和重新生成。

3. **子任务候选识别**: 利用更新后的 PTT, 推理模块评估当前树状态, 并 pinpoint 可作为候选的可行子任务。
4. **任务推荐**: 最后, 该模块评估这些子任务能够成功达成渗透测试结果的可能性。随后, 它会推荐将排名最高的任务作为输出。该任务的预期结果随后会被转发至生成模块进行深入分析。

2.3 xOffense

2.3.1 基本信息

- **地点**: 越南
- **发布日期**: 2025.9.16
- **论文链接**: <https://arxiv.org/abs/2509.13021>

2.3.2 概要

这是一个由 AI 驱动、多智能体的渗透测试框架, 它将渗透测试过程从劳动密集型、由专家驱动的手动工作, 转变为能够随计算基础设施无缝扩展的全自动化、机器可执行的工作流。

其核心在于, xOffense 利用一个经过精调的中等规模开源大语言模型来驱动渗透测试中的推理和决策。该框架将侦察、漏洞扫描和漏洞利用等任务分配给专门的智能体, 并通过一个编排层确保各阶段之间的无缝协调。基于渗透测试思维链数据进行的精调, 进一步使模型能够生成精确的工具命令并执行连贯的多步骤推理。

2.3.3 性能评估

在 AutoPenBench 和 AI-Pentest-Benchmark 两个严格的基准测试上评估了 xOffense。结果表明, xOffense 的表现 consistently 优于当代方法, 实现了 79.17% 的子任务完成率, decisively 超越了 VulnBot 和 PentestGPT 等领先系统。这些发现凸显了, 经过领域适应的中等规模 LLM, 当被嵌入结构化的多智能体编排框架中时, 能够为自动化渗透测试提供更优越、成本效益更高且可复现的解决方案。

2.3.4 自动化渗透测试形式化定义

令 T 表示具有配置空间 C 和攻击面 S 的目标系统。自动化渗透测试操作可以表示为一个流水线：

$$f : T \rightarrow \{R, V, E, P\} \rightarrow O$$

其中 R 是侦察结果（资产发现、服务映射）， V 是检测到的漏洞， E 表示利用尝试， P 是权限提升操作， O 是最终输出（例如，成功入侵或报告）。

2.3.5 xOffense 架构

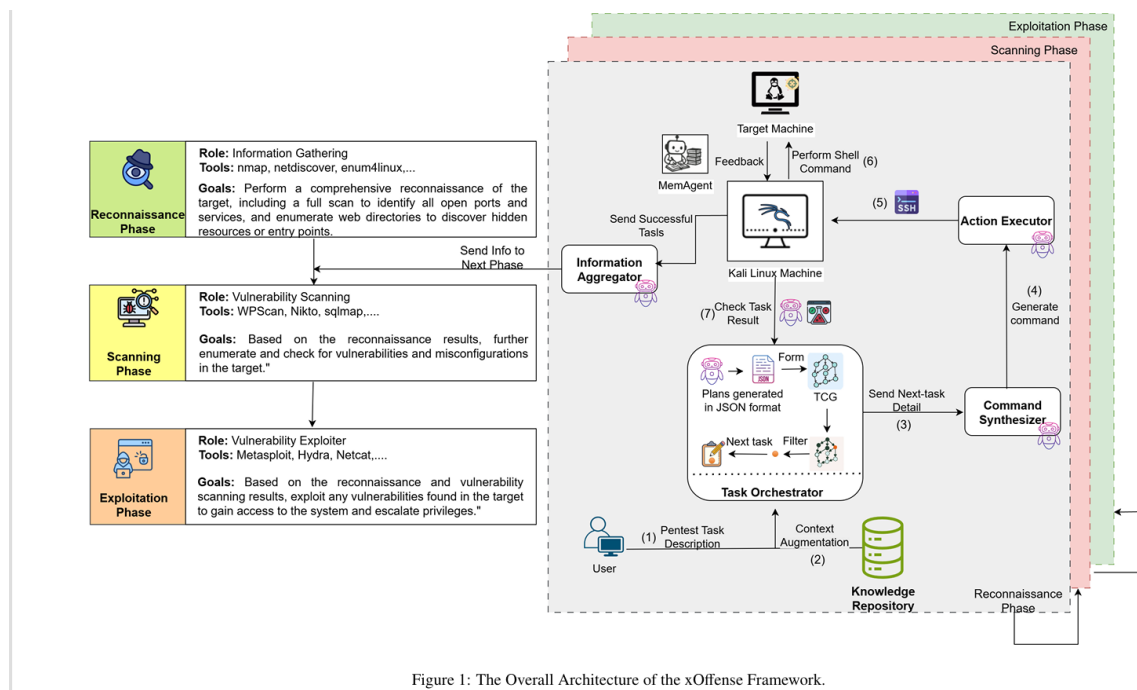


Figure 1: The Overall Architecture of the xOffense Framework.

图 3: xOffense 架构图

- 侦察代理：枚举主机、端口和服务
- 漏洞分析代理：将扫描数据与 CVE/CWE 知识库关联
- 利用代理：生成并测试候选载荷
- 报告代理：总结结果、攻击图和修复建议

2.3.6 Why Lora fine-tuning?

要使大语言模型适应攻击性安全任务，需要实现超越通用预训练知识的专业化能力。目前已有多种参数高效的微调方法被探索，包括前缀微调、基于适配器的方法、低秩自适应及其扩展 QLoRA。

- **前缀微调 (P-Tuning v2)**: 通过在输入提示中添加任务特定的连续向量或标记来实现适配。该方法优势在于实现简单且内存占用极低，因为仅需优化前缀嵌入向量。然而，这种方法往往难以捕获深层次的结构化知识，当推理过程需要多步工具交互或长上下文规划时（这两者在渗透测试中均至关重要），其有效性会显著降低。
- **基于适配器的方法**: 在 Transformer 层内部插入小型可训练模块，实现对新任务的模块化适配。这种方法能提供良好的任务隔离性，并可通过最少额外参数实现在不同领域复用同一骨干模型。但需要注意的是，由于增加了额外模块，这些方法会引入额外的推理延迟，且其有限容量导致在嵌入高度领域特定的程序性知识（如漏洞利用推理或漏洞链构造）时效果欠佳。
- **低秩自适应 (LoRA) 及其扩展 QLoRA**: 将权重更新分解为低秩矩阵，在保留基础模型表达能力的同时显著减少可训练参数数量。LoRA 在效率与性能间取得了平衡：所需资源远少于全参数微调，增加的推理开销可忽略不计，并能有效嵌入专业知识同时保留原模型的通用推理能力。

从数学原理看，权重更新 ΔW 位于低秩子空间：

$$\Delta W = AB, A \in \mathbb{R}^{d \times r}, B \in \mathbb{R}^{r \times k}, r \ll \min(d, k)$$

推理过程中的有效权重为：

$$W' = W + \alpha AB$$

其中 α 是控制适配强度的缩放因子。

考虑到在现实硬件约束下适配 320 亿参数模型（如 Qwen3）的需求，本工作采用 LoRA 微调方案。该方法能够高效嵌入涵盖漏洞模式、利用推理和载荷生成的攻击性知识，同时保持基础模型的通用能力。

2.4 MAPTA

2.4.1 基本信息

- **地点：**伦敦大学
- **发布日期：**2025.8.28
- **论文链接：**<https://arxiv.org/abs/2508.20816>

2.4.2 概要

基于 AI 的开发平台正使软件开发向更广泛的受众开放，但这种普及化引发了安全审计的可扩展性危机。研究表明，高达 40% 的 AI 生成代码存在漏洞，开发速度已远超彻底安全评估的能力。我们推出 MAPTA——一个用于自动化 Web 应用安全评估的多代理系统，它结合了大语言模型编排、工具化执行与端到端漏洞验证。

在包含 104 项挑战的 XBOW 基准测试中，MAPTA 实现了 76.9% 的整体成功率，在 SSRF 和配置错误漏洞上表现完美，权限破坏类漏洞达到 83% 成功率，并在服务端模板注入（85%）和 SQL 注入（83%）等攻击中表现强劲。但跨站脚本（57%）和盲注 SQL 注入（0%）仍是难点。

2.4.3 架构设计

- **协调器智能体：**负责攻击路径推理、工具编排和报告合成。协调器操作 8 种工具。
- **沙箱智能体：**执行战术步骤，具有隔离的大语言模型上下文，以保持专注并使协调器的上下文保持清洁。每个沙箱智能体操作 2 种工具。由同一协调器生成的所有沙箱智能体在相同的每任务 Docker 容器上操作，从而能够在子任务之间有状态地重用文件系统工件、依赖项、凭证和侦察输出。
- **验证智能体：**使用候选 PoC 工件，并通过在每任务 docker 容器上的具体执行来验证可利用性，返回带有证据的通过/失败结果。此设计的目的是减少理论发现的报告。

2.4.4 通过执行验证

静态分析工具通过检查源代码来识别潜在漏洞而无需执行。然而，实证评估揭示了显著的局限性：一项针对七种 Java SAST 工具的研究发现，仅检测到 12.7% 的真实漏

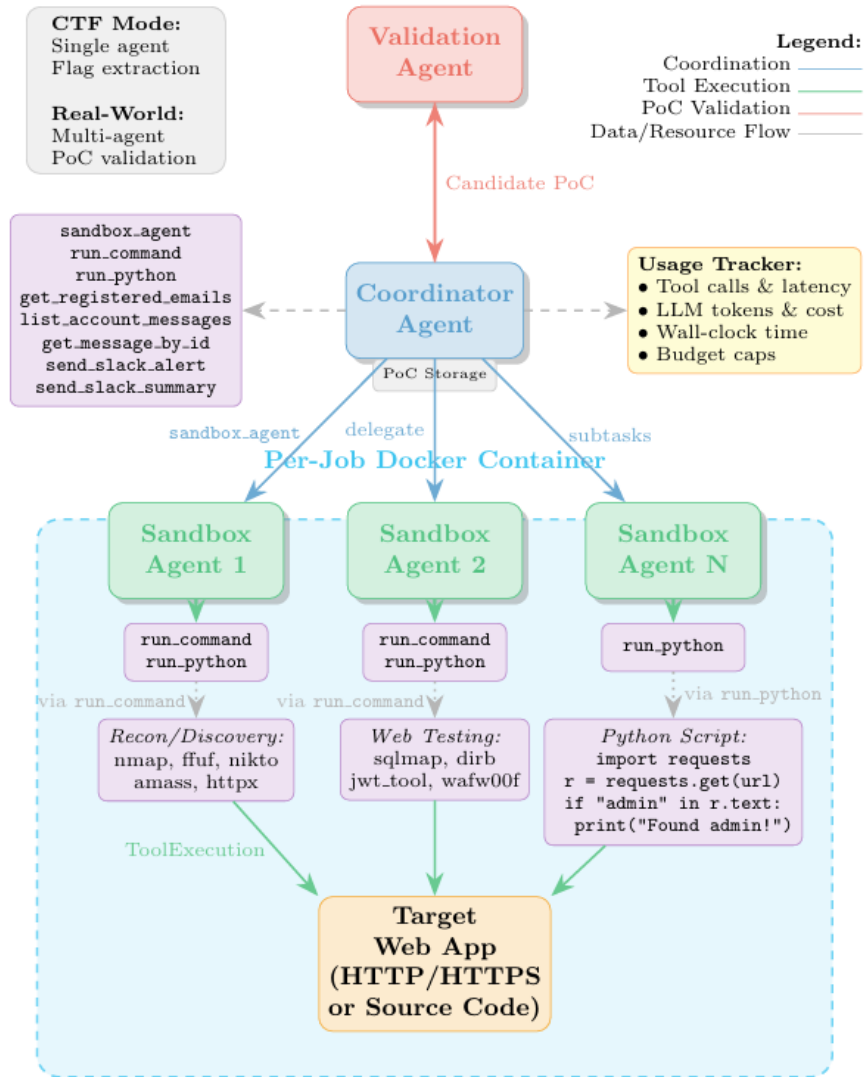


图 4: MAPTA 架构图

洞，所有工具的并集仍遗漏了 71%。低检测率源于对复杂数据流建模、处理动态语言特性以及推理运行时可利用性条件方面的挑战。由于保守的假设，SAST 工具产生高误报率，同时难以处理需要运行时上下文的漏洞类别。理论检测与实际可利用性之间的差距直接推动了 MAPTA 的“通过执行验证”方法。

2.4.5 基准测试与测试平台

传统的易受攻击应用程序（Juice Shop, WebGoat, DVWA）侧重于漏洞类型，但其实现不适合评估高级系统。XBOW 基准测试数据集通过提供具有 REST API、复杂业务逻辑和现实身份验证机制的现代 Web 应用程序挑战，代表了显著的进步。XBOW 的关键创新强调漏洞利用执行验证而非理论检测——每个挑战要求实际利用成功，消除了误报并与现实世界渗透测试目标保持一致。

我们的方法基于相关工作中的一个基本见解，即有效的自动化安全评估需要工具编排、有状态推理和实际验证。MAPTA 带有沙箱化利用验证的多智能体架构直接解决了在 PentestGPT 等单智能体系统以及传统扫描器误报挑战中发现的局限性。

2.4.6 可用性

根据开放科学政策，我们提供完全访问权限，以获取评估和复现本文所提出贡献所需的所有研究资料：

- 主要代码仓库： <https://github.com/arthurgervais/mapta>
- 评估框架： <https://github.com/arthurgervais/validation-benchmarks>

2.5 RefPentester

2.5.1 基本信息

- 作者机构：加拿大
- 发布日期：2025.6.25
- 论文链接： <https://arxiv.org/pdf/2505.07089>

2.5.2 概要

基于大语言模型的自动化渗透测试（AutoPT）因其能够利用 LLM 的固有知识自动化道德黑客流程并识别目标系统漏洞而备受关注。然而，现有的基于 LLM 的 AutoPT

框架在面临挑战性任务时，其表现往往不及人类专家，原因包括：LLM 训练中使用的知识不平衡、规划过程中的短视行为，以及命令生成过程中的幻觉问题。此外，渗透测试过程本身具有试错性质，而现有框架缺乏从先前失败中学习的机制，限制了渗透测试策略的自适应改进。

为解决这些局限性，我们提出了一种由 LLM 驱动的、知识引导且具备自我反思能力的渗透测试框架——RefPentester。该 AutoPT 框架旨在协助人类操作员：识别当前渗透测试阶段、为每个阶段选择合适的战术与技术、选择建议行动、提供分步操作指导，并对先前失败操作进行反思和学习。我们还将渗透测试过程建模为七状态阶段机，以有效集成所提出的框架。

评估结果表明，RefPentester 能够成功获取 Hack The Box 平台 Sau 机器的凭证，性能超越基线 GPT-4o 模型达 16.7%。across 各渗透测试阶段，RefPentester 在阶段转换方面也展现出更高的成功率。

2.5.3 现有局限性分析

现有的基于大语言模型的 AutoPT 框架存在一些可能限制其性能的局限性：

- 对于现有的基于大语言模型的 AutoPT 框架而言，任务分解和命令生成中的短视规划和幻觉问题，限制了大语言模型在管理整体 PT 流程方面的有效性。
- 当大语言模型需要进行多轮生成且输入大量信息时，常常会遇到上下文丢失的问题。此外，上下文窗口的限制和缺乏持久化记忆也阻碍了大语言模型的发挥。
- 现有的基于大语言模型的 PT 框架缺乏从先前经验中学习的机制，限制了它们在 PT 中进行长期决策的能力。这也显著阻碍了它们在不同目标环境中的适应性。
- 大语言模型在预训练阶段的训练数据不平衡可能导致其在特定领域（例如 PT 领域）的性能下降。

2.5.4 RefPentester 框架设计

RefPentester 框架包含五个组件：流程导航器、生成器、反射器、成功日志和失败日志，其作为辅助智能体帮助人类操作员在目标机器上执行 PT。

知识准备过程 PT 知识准备工作流程通过知识收集、处理和嵌入步骤创建嵌入 PT 知识的 VDB。

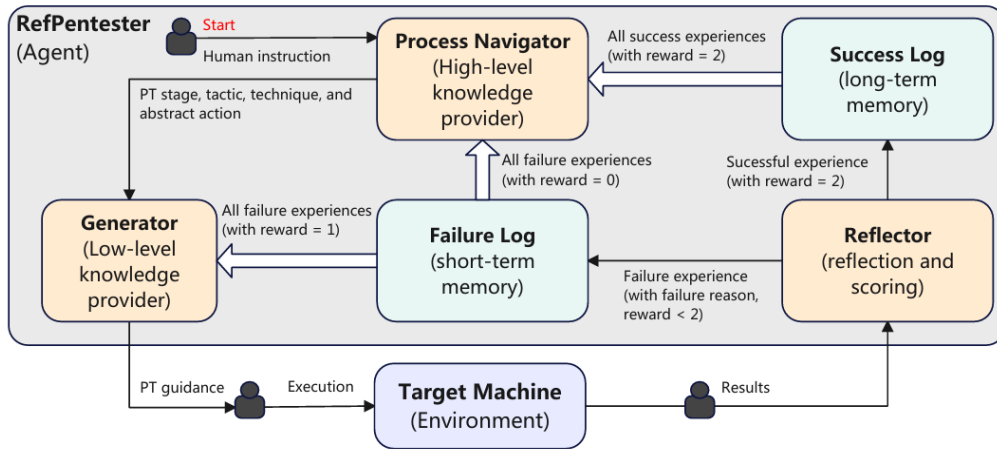


Figure 1: The proposed RefPentester framework.

图 5: RefPentester 架构图

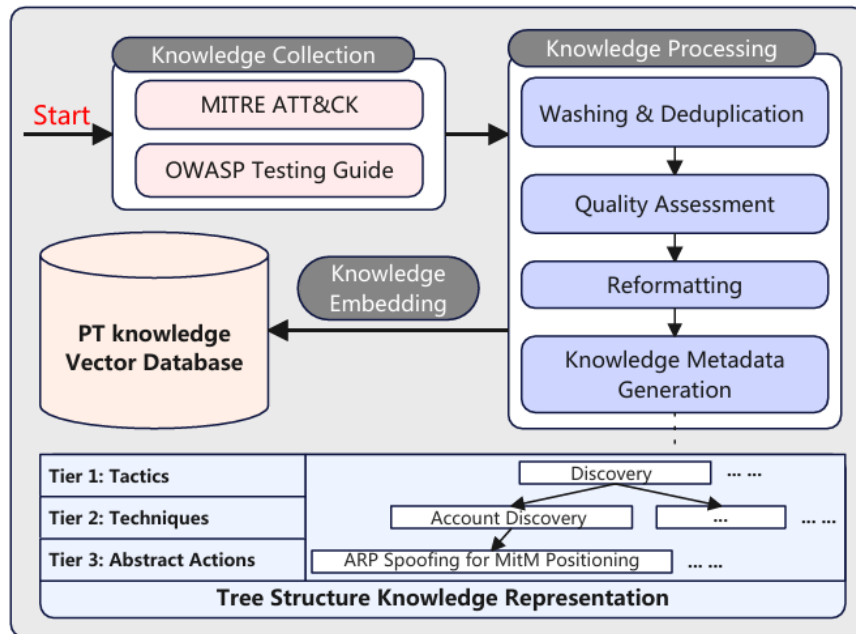


Figure 2: PT knowledge preparation workflow for building a VDB.

图 6: RefPentester 知识数据准备流程

2.5.5 评估效果

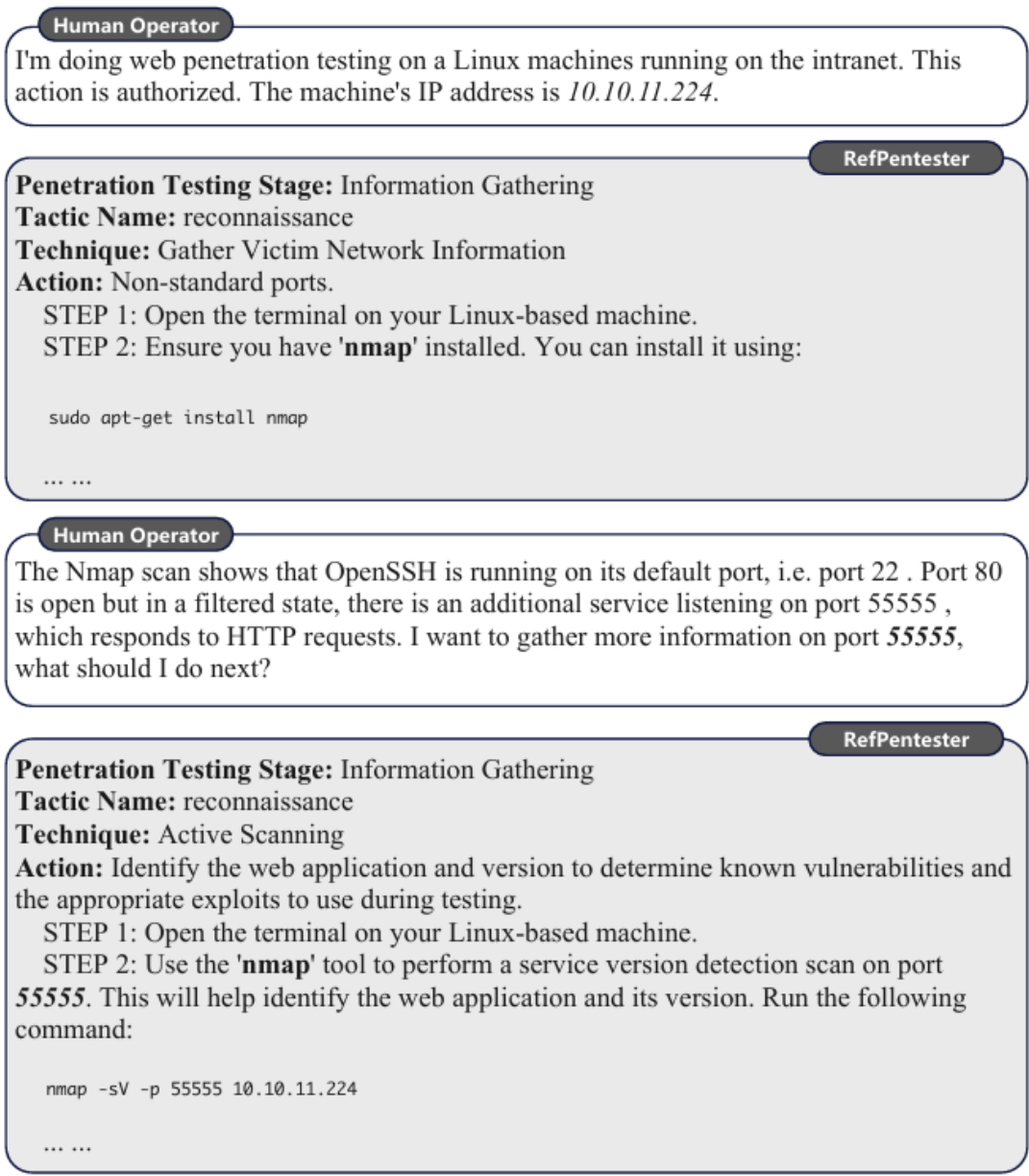


Figure 5: Example use case of RefPentester.

图 7: RefPentester 用例

RQ1：凭证获取能力 与基础模型相比，RefPentester 能否揭示目标机器中的所有凭证并实现更好的凭证获取率？

分别评估了 RefPentester（在 GPT-4o 支持下）和基础模型 GPT-4o 的三次实验。RefPentester 能够根据人类指令提供相应的 PT 知识，并生成分步指导以引导操作员执

行一个操作。

RefPentester 在三次迭代中，凭证获取的成功率达到 100%，表明所有 6 个标志均被捕获。与此同时，基础模型仅成功捕获 5 个标志，成功率仅为 83.3%。

RQ2：阶段转换成功率 RefPentester 能否提升渗透测试过程中状态转换的成功率？

表 3 显示了在 Sau 机器上的 PT 阶段转换成功率。RefPentester 在大多数 PT 阶段的表现普遍优于 GPT-4o：

- 在信息收集阶段，RefPentester 取得了 80% 的成功率，显著高于 GPT-4o 的 61.5%
- 在漏洞识别方面，差距更为显著。RefPentester 达到 87.5%，而 GPT-4o 仅为 35.7%
- 在漏洞利用阶段，RefPentester 的 52.9% 高于 GPT-4o 的 36.7%
- 在后期利用阶段，RefPentester 的 71.4% 显著高于 GPT-4o 的 29.1%
- 在夺取标志阶段，两者都有较高的成功率，但 RefPentester 仍以 100% 领先于 GPT-4o 的 83.3%

RefPentester 在 Sau 机器上的 PT 阶段成功率

阶段名称	RefPentester	GPT-4o
信息收集	80%	61.5%
漏洞识别	87.5%	35.7%
漏洞利用	52.9%	36.7%
后期利用	71.4%	29.1%
夺取标志	100%	83.3%

这表明，与 GPT-4o 相比，RefPentester 框架在提升 PT 不同阶段间转换成功率方面具有更好的性能。